EEE 475/575 Medical Image Reconstruction & Processing
Homework #3
15 April 2026, Wednesday at 11:59 -- Noon

GUIDELINES FOR HOMEWORK SUBMISSION

- Upload your solutions to Moodle as **two separate files**:
  1) **PDF Report File:** The report should be typeset (i.e., no handwriting) and should properly present your results. In addition, the report should display the relevant sections of your code at the end of each subpart. No partial credits will be given to answers with missing codes or unjustified answers.
     At the beginning of your report, provide a section titled "GenAI Usage Disclosure" and clearly explain whether you have used any genAI tools. Include the name/version of the tool and a brief description of how it was used (see the "GenAI Policy" included below).
  2) **Code File:** You can use any programming platform of your choice. If you do not upload your code file, your homework will not be evaluated (i.e., 0 pts).
- Submission system will remain open for 1 day after the deadline:
  - No points will be lost if you submit your assignment within 12 hours past the deadline.
  - There will be a 50% penalty if you submit after 12 hours but within 24 hours past the deadline.
  - No submissions beyond 24 hours past the deadline.

GenAI Policy: Generative AI (genAI) tools (e.g., large language models, coding assistants) may be used for homework assignments and project to assist with coding and to improve writing. Any use of genAI must be clearly disclosed, including the name/version of the tool and a brief description of how it was used. Upon request, the student should be ready to provide prompts, outputs, and execution logs pertaining to their genAI use for all assignments. Students remain fully responsible for all submitted material and must be able to explain and justify their code, results, and written content. Reliance on genAI does not transfer responsibility for correctness or originality. Failure to disclose AI usage or inappropriate reliance on AI-generated content will be treated as a violation of academic integrity.

PREPARATION

From the Moodle page of the class, download the following files:

- **SM_25nm_50x50.mat:** System matrix calibration measurements for a simulated MPI system for magnetic nanoparticles 25 nm core diameter, with N×N=50×50 grid size and 2 receive coils, scanned using a Lissajous trajecrtory. This file contains a single 3-D array SM with size 2×1632×2500, containing the Fourier transform of the signal for each calibration measurement. Here, the first index is for receive coils, the second index the frequency axis, and the third index is for position (i.e., N×N =50×50 = 2500). For example, SM(1,:,5) contains the calibration measurement from Coil #1 when a point source object was placed at 5^{th} grid location. This SM does not contain any noise.
- **meas_25nm_phantom.mat:** Measurement data for a simulated phantom. This file contains two 2-D arrays (phantom_ref and meas). phantom_ref has size N×N=50×50 and contains phantom for which the measurement data was simulated. meas has size 2×1632 and contains the Fourier transform of the simulated signal from the phantom, scanned using the same Lissajous trajectory that was used in the calibration measurement. Here, the first index is for receive coils and the second index the frequency axis. This simulated measurement does not contain any noise.
- **su_openmpi.mat:** This file contains *S* and *u* vectors from an actual experimental, downloaded from OpenMPI dataset (https://magneticparticleimaging.github.io/OpenMPIData.jl/latest/). This data belongs to a 3D Lissajous trajectory with 37×37×37 grid size, where the signal is acquired by two different coils. The system matrix *S* has size 3056×50653, where the first index is for frequency, and the second index is for position (i.e., 37×37×37 = 50653). The measurement vector *u* has size 3056×1.

PART I – SYSTEM MATRIX (10 pts)

1.1) Viewing Calibration Data: Load SM_25nm_50x50.mat. This file contains system matrix calibration measurements in frequency-domain. Working in frequency-domain is more advantageous in MPI, as the signal is sparse in frequency domain. For both coils, plot the magnitude spectrum for calibration measurements at three different grid locations. For example, for Coil #1 and 5^{th} grid location,

```
>> plot(abs(SM(1,:,5)))
```

As you can see in the spectrum plots, non-centered Fourier transform was performed on the signal, such that the first point in the spectrum corresponds to DC. The spectrum contains signal at certain discrete frequencies. These are the higher harmonics of the drive field frequencies that was used to induce the nanoparticle signal in MPI.

1.2) Preparing the System Matrix: Next, reformat SM to form the system matrix *S*. Do the following: The n^{th} column of *S* should contain the following:

1) Since the time-domain calibration signal is real valued, the spectrum has conjugate

symmetry property. Therefore, only half of the spectrum contains "nonredundant" information. Therefore, discard the second half of the frequency-domain data:

```
>> S = SM(:,1:end/2,:);
```

Now, *S* will have size 2×816×2500.

2) Reshape *S* to concatenate the calibration measurements from the two coils along the frequency axis:

```
>> S = reshape(S,2*816,2500);
```

Now, *S* will have size 1632×2500.

3) We know that the MPI image shows the nanoparticle concentration in each grid location, which is real-valued. Therefore, it makes things easier if the system matrix is also real-valued. For this purpose, concatenate the real and imaginary parts of *S* along the frequency axis:

```
>> S = [real(S); imag(S)];
```

In the end, *S* has size 3264×2500 and is purely real-valued. Each column of *S* contains the calibration data for a given grid position. Plot <u>three different columns</u> of the system matrix. For example, for 5th grid location:

```
>> plot(S(:,5))
```

Save the system matrix *S* in a .mat file, to be used in the following questions. What is the size of this .mat file?

1.3) Preparing the Measurement Vector: Load meas_25nm_phantom.mat. This file contains frequency-domain measurements for a simulated phantom. First, display the phantom that is given as a matrix with size 50×50. This phantom shows the spatial distribution of nanoparticles in the FOV, where the pixel intensity denotes nanoparticle concentration. We will use this image as the "*<u>reference</u>*" image, m_{ref}, for the rest of this homework. Define a variable named max_val as the maximum pixel intensity in m_{ref}.

Prepare the measurement vector *u* following identical steps as in Part (1.2):

```
>> u = meas(:,1:end/2);
>> u = reshape(u,2*816,1);
>> u = [real(u); imag(u)];
```

In the end, *u* should have size 3264×1. It is very important that *u* and the columns of *S* are prepared using identical steps. Plot the resulting measurement vector *u*. Save the measurement vector *u* in a .mat file, to be used in the following questions. What is the size of this .mat file?

<u>IMPORTANT NOTE</u>: In Part II and Part III, use an *imshow* grayscale window of [0 max_val] for all images and grayscale window of [0 0.5*max_val] for all error images to be able to compare the results across different methods. During PSNR and SSIM computations, normalize both the image of interest and m_{ref} by max_val computed from m_{ref}.

PART II –SVD (35 pts)

2.1) SVD: Compute the compact Singular Value Decomposition (SVD) of S as follows:

```
>> [U,Sigma,V] = svd(S,'econ');
```

Here, Sigma is a diagonal matrix of size 2500×2500 that contains the singular values of *S*. *U* is of size 3264×2500 and *V* is of size 2500×2500.

Plot the singular values (i.e., the diagonal entries of Sigma). Note that the singular values are ordered from largest to smallest. Compute the condition number of *S* using these singular values, i.e., $Cond(S) = \frac{\sigma_{max}}{\sigma_{min}}$.
Hint: Check that you get the same condition number using the built-in *cond* function of MATLAB.

2.2) Image Reconstruction via SVD: Compute the image *c* using Moore-Pensore pseudo inverse from SVD, i.e., $c = V\Sigma^{-1}U^{H}u$. Here, *c* will be a vector of size 2500×1. Reshape this vector into a 2D image of size 50×50:

```
>> ima = reshape(c,50,50);
```

Note that MPI images cannot have negative values, since the pixel intensity represents the nanoparticle concentration. Therefore, set all of the negative pixel values in the image to zero at the last step (**do this for ALL reconstructed images in this homework**):

```
>> ima(ima<0) = 0;
```

Display the resulting image. Display the error image (i.e., the magnitude of the difference between the reconstructed image and the reference image). Compute PSNR and SSIM. Briefly comment on the problems that you see in the image.

2.3) Truncated SVD: Truncate the singular values such that the condition number is approximately 100. Accordingly, you will keep the largest M singular values, such that $\frac{\sigma_1}{\sigma_M} \approx 100$. You will then keep the first M columns of U and V. In the end, truncated version of Sigma will have size M×M, truncated version of U will have size 3264×M, and truncated version of V will have size 2500×M.

Using these truncated versions, compute the image following the same steps as in Part (2.2). Display the image and the error image. Compute PSNR and SSIM. Briefly comment on the improvements on the reconstructed image.

2.4) Repeat Part (2.3) for the following condition numbers: [1 5 10 10^2 10^3 10^4 10^5 10^6 10^7]. At each condition number, display the image and error image, compute PSNR and SSIM. Next, plot PSNR and SSIM as functions of condition number (you may want to use a logarithmic x-axis to better see the results). Briefly comment on the image quality as a function of condition number. Among the tested values, what is the condition number that yields the best image quality?

2.5) Filtered SVD: We will now filter the singular values using the filter values:

$$y_i = \frac{\sigma_i^2}{\sigma_i^2 + \lambda}$$

where λ is a regularization parameter. This is equivalent to replacing the singular value σ_i by the filtered singular value ${\sigma_i}'$:

$${\sigma_i}' = \frac{\sigma_i}{y_i} = \frac{\sigma_i^2 + \lambda}{\sigma_i} = \sigma_i \left(1 + \frac{\lambda}{\sigma_i^2}\right)$$

Importantly, solving the filtered SVD case is equivalent to solving the following regularized least squares problem:

$$(S^H S + \lambda I)c = S^H u$$

<u>Show this</u> by inserting the SVD decomposition $S = U\Sigma V^H$ in to the equation above and deriving the following relation:

$$c = V\tilde{\Sigma}^{-1} U^H u$$

where $\tilde{\Sigma}$ is a diagonal matrix with the following diagonal entries corresponding to $\tilde{\Sigma}_{i,i} = {\sigma_i}'$.

2.6) The equivalency of filtered SVD and regularized least squares problem give us an insight into the reasonable range of values for the parameter λ. Here, λ should be such that $\lambda \ll \sigma_i^2$ for the largest singular values, so that it does not affect them. On the other hand, λ should be such that $\lambda \gg \sigma_i^2$ for the smallest singular values, so that the inversion of the small singular values does not cause noise amplification.

To start with, choose $\lambda = \sigma_1 \sigma_N$ (i.e., the product of the smallest and largest singular values). Plot the filtered singular values. Also plot on the same axis the original singular values for comparison. Compute the image for filtered SVD using this λ. Display the image and error image. Compute PSNR and SSIM. Briefly comment on the result.

2.7) Try a few different λ values in the reasonable range. Plot PSNR and SSIM as functions of λ. Use PSNR and SSIM, as well as visual inspection, to determine an optimal λ^* (approximately). What is λ^* (approximately)? **Hint:** You may want to try powers of 10 in the reasonable range so that you can capture the trends in PSNR and SSIM as a function of λ.

Then, display the following results for the cases of $\lambda^*/10$, λ^*, and $10\lambda^*$: the image, error image, PSNR and SSIM. Briefly comment on the image quality as a function of λ.

2.8) L-curve: In a real-life imaging scenario, we do not know the "*reference*" image, so λ^* cannot be determined using PSNR or SSIM. One method to determine λ^* is to plot the L-curve for residual norm vs. solution norm (i.e., $\|Sc - u\|_2$ vs. $\|c\|_2$). The optimal value for λ is typically the one that yields a result at the corner of this L-curve. Plot this L-curve by trying a few different λ values in the reasonable range. Make sure you capture a decent part of the L-curve and not just a small section of it (again, you may want to try powers of 10 in the reasonable range for this). What is λ^* (approximately) according to the L-curve? For this λ^*, display the image, error image, PSNR and SSIM. Briefly comment on the results.

PART III – KACZMARZ METHOD (35 pts)

3.1) Row-norm Thresholding: Standard Kaczmarz inherently normalizes the rows of the system matrix. During that normalization, rows that have close to zero energy can cause noise amplification problems. To avoid this problem, we will first perform thresholding to remove the frequency components (i.e., the rows) of the system matrix with small energy. For this, first compute the norm of each row of the system matrix *S*. Plot the norms (i.e., row number vs. row norm). Define a variable named max_norm as the maximum row norm. Then, discard the rows of S that have norm smaller than $max\_norm\ /10$. Likewise, make sure to discard the corresponding entries of u, so that you keep the identical frequency components in u. What are the sizes of *S* and u after this thresholding?

3.2) Standard Kaczmarz Method: Use the thresholded matrices and vectors from Part I. Implement standard Kaczmarz method[1], without any regularization or relaxation. Make sure that you <u>randomize</u> the order of the rows during sub-iterations (**Hint:** You may want to use the built-in *randperm* function of MATLAB). At the end of each iteration (NOT sub-iteration), set all of the negative entries of the reconstructed concentration vector c to zero.

For the outputs of the first 10 iterations (NOT sub-iterations) of standard Kaczmarz, display the image and error image. Plot PSNR and SSIM as a function of iteration count. Briefly comment on the image quality as a function of iteration count. Note that because of randomization in row order, you will get slightly different results each time you run the algorithm.

3.3) Reconstruct the image using standard Kaczmarz with the following row-norm thresholds: max_norm*[10^{-4} 10^{-3} 10^{-2} 10^{-1}]. For the output of only the 10^{th} iteration of each case, display the image and error image. Plot PSNR and SSIM as a function of row-norm threshold (use 10^{th} iteration results only). Briefly comment on the image quality as a function of row-norm threshold.

3.4) Regularized Kaczmarz Method: Regularized Kaczmarz method can further benefit from row energy normalization. However, we will ignore that fact for the purposes of this homework. For this question, do not apply any row-norm thresholding. Implement the regularized Kaczmarz method[2], incorporating a regularization parameter λ. Do not use any weighting matrix.

To start with, choose $\lambda = \sigma_1 \sigma_N$. Again, make sure that you <u>randomize</u> the order of the rows during sub-iterations. For the outputs of the first 10 iterations of regularized Kaczmarz, display the image and error image. Plot PSNR and SSIM as a function of iteration count. Briefly comment on the image quality.

3.5) One approach to choose λ is to define it relative to σ_1^2, i.e., $\lambda = \sigma_1^2\, \lambda_{rel}$. This way, we know that we should choose $0 < \lambda_{rel} < 1$. Reconstruct the image using regularized Kaczmarz for λ_{rel} = [10^{-6} 10^{-5} 10^{-4} 10^{-3} 10^{-2}]. For the output of the 10^{th} iteration of each case, display the image and error image. Plot PSNR and SSIM as a function of λ_{rel} (use 10^{th} iteration results only). Briefly comment on the image quality as a function of λ_{rel}.

---

[1] See the slides with title "Kaczmarz Method" on "Linear System of Equations" slides.

[2] See the slides with title "Alternative Kaczmarz Method for Regularized Weighted Least Squares Problem" on "Linear System of Equations" slides. Also see Algorithm 2 on Page#145 of the Book Chapter posted on Moodle.

3.6) Effects of Measurement Noise: So far in this homework, the measurement vector u did not contain any noise. Add white Gaussian noise with zero mean and standard deviation $max\_norm/1000$ to the entries of the measurement vector u. Do not add noise to the system matrix, as we are assuming that it was acquired in a high SNR setting. Then, repeat Part (3.3) for this case. Briefly comment on the image quality as a function of row-norm threshold in the presence of noise.

3.7) For the same noise added vector u, repeat Part (3.5). Briefly comment on the image quality as a function of λ_{rel} in the presence of noise.

PART IV – OPEN MPI DATA (20 pts)

4.1) In the previous parts, we used a simulated dataset. Here, we will work with an actual experimental MPI dataset, downloaded from the Open MPI database [3]. Load su_openmpi.mat. The system matrix and measurement vector have already been preprocessed and prepared in proper format. Plot the singular values of the system matrix *S*. Note that the size of *S* is relatively large, so loading the .mat file and computing the SVD may take a while. Firstly, plot three different columns of the system matrix, as in Part (1.2).

4.2) Compute the compact SVD of *S*. Plot the singular values and compute the condition number of *S* using these singular values.

4.3) Reconstruct the image using regularized Kaczmarz with $\lambda = \sigma_1 \sigma_N$, without row-norm thresholding. Here, the reconstructed vector c needs to be reshaped into a 3D image volume of size 37×37×37:

```
>> ima = reshape(c,37,37,37);
```

At the end of the 10th iteration, display all 37 slices of the reconstructed image (assume that the third index of ima is the slice index). We can display all 37 slices as a "montage" of images (i.e., concatenated slices) by using the following command:

```
>> montage(reshape(ima,[37, 37, 1, 37]),'displayRange',[]);
```

4.4) Reconstruct the image using regularized Kaczmarz for λ_{rel} = [10^{-6} 10^{-5} 10^{-4} 10^{-3} 10^{-2}], where $\lambda = \sigma_1^2\, \lambda_{rel}$. For the output of the 10th iteration of each case, display all 37 slices of the reconstructed image in montage format. Here, we do not have a ground truth reference image. Based on your visual evaluation, which of these λ_{rel} values gave the best image quality? Briefly comment on the image quality as a function of λ_{rel} .

[3] https://magneticparticleimaging.github.io/OpenMPIData.jl/latest/datasets.html